

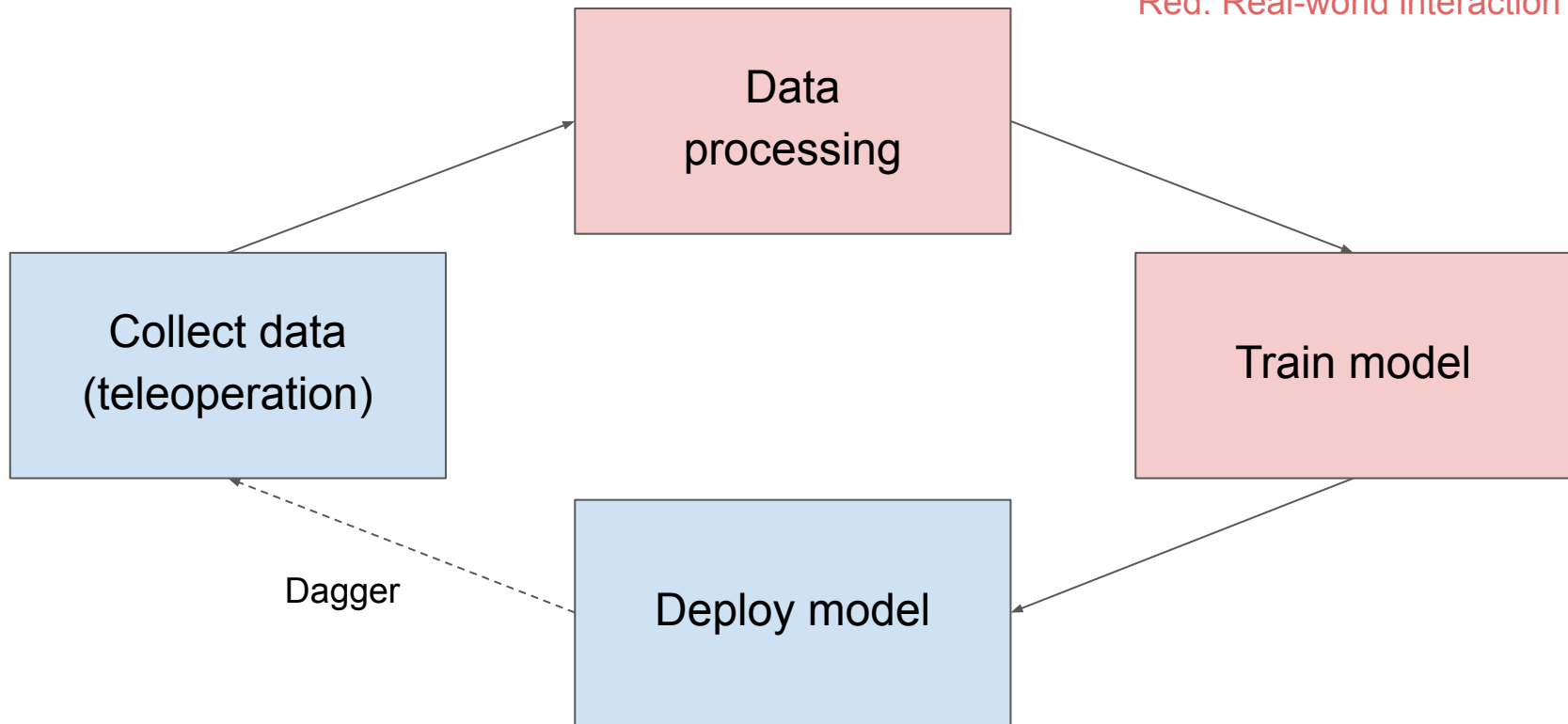
Imitation Learning Tutorial

Neuromeka AI team

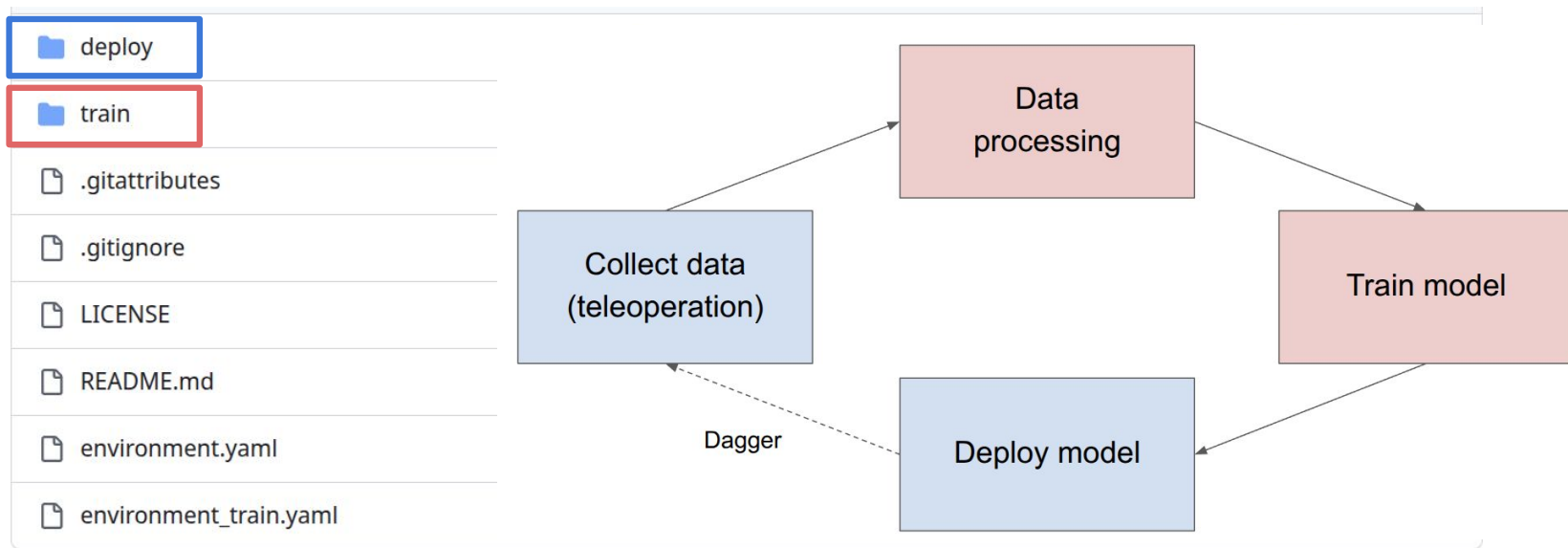
Imitation learning pipeline

Blue: Real-world interaction O

Red: Real-world interaction X



Code structure



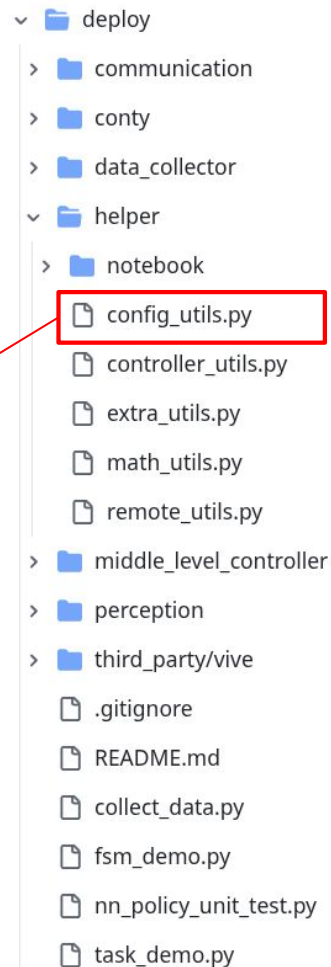
Setting up configurations for real-world

1. ROBOT_CONFIG

2. TASK_CONFIG

- a. Task name
- b. CAMERA_CONFIG
- c. MODEL_CONFIG
- d. DATA_CONFIG
- e. EXTRA_CONFIG

- **Base configuration** is located here.
- **Create a new instance** from it to generate new configurations.



Setting up configurations for real-world

1. ROBOT_CONFIG

- Robot configuration
- Ex) robot ID, robot IP, home position (joint-space), control parameters, third-party gripper ..

- Robot ID
- Used to distinguish multiple robots

```
CUSTOM_ROBOT_CONFIG = ROBOT_CONFIG(  
    robot_params = {  
        0: {  
            "ip": "192.168.0.147",  
            "home_pos": [0., 0, -90., 0., -90., 0.],  
            "gripper": {  
                "enable": False,  
                "type": "RobotiqUSBCClient",  
                "params": {  
                    "port": "/dev/robotiq_2f85"  
                }  
            },  
            "control": {  
                "vel_scale": 1., # 0 ~ 1  
                "acc_scale": 10., # 0 ~ 10  
                "move_vel_scale": 50., # 0 ~ 100  
                "move_acc_scale": 50. # 0 ~ 1000  
            }  
        },  
    },  
    control_dt = 0.05  
)
```

Setting up configurations for real-world

1. ROBOT_CONFIG

2. TASK_CONFIG

- Task name
- CAMERA_CONFIG
- MODEL_CONFIG
- DATA_CONFIG
- EXTRA_CONFIG

```
@dataclass
class TASK_CONFIG:
    name: str = "base"
    camera_config: CAMERA_CONFIG | None = CAMERA_CONFIG()
    model_config: MODEL_CONFIG | None = MODEL_CONFIG()
    data_config: DATA_CONFIG | None = DATA_CONFIG()
    extra_config: EXTRA_CONFIG = EXTRA_CONFIG()
```

- Task name

Setting up configurations for real-world

1. ROBOT_CONFIG

2. TASK_CONFIG

- Task name
- CAMERA_CONFIG
 - Camera configuration
 - Ex) camera name, serial number, camera parameters ..
 - Supports realsense camera

```
camera_config = CAMERA_CONFIG(  
    cam_params = {  
        "left": {  
            "serial": "207222072747",  
            "exposure": 50,  
            "enable_depth": False  
        },  
        "right": {  
            "serial": "317622073859",  
            "enable_depth": False  
        }  
    }  
),
```

- Camera name

Setting up configurations for real-world

1. ROBOT_CONFIG

2. TASK_CONFIG

- Task name
- CAMERA_CONFIG
- MODEL_CONFIG
 - Imitation learning model configuration
 - Ex) model type, model weight path ..

```
model_config = MODEL_CONFIG(  
    model_type = "act",  
    model_dir = "/home/nrmk/neuromeka-il/train/weights/broom/2025-08-21-21-31-21",  
    model_file = "policy_last.ckpt",  
    success_threshold = 0.8,  
    device = "cuda"  
)
```


Setting up configurations for real-world

1. ROBOT_CONFIG

2. TASK_CONFIG

- Task name
- CAMERA_CONFIG
- MODEL_CONFIG
- DATA_CONFIG
 - Data collection configuration
 - Ex) path to save data, path to save visualization data, teleoperation device, device parameters ..

```
data_config = DATA_CONFIG(  
    data_dir = os.path.join(get_base_dir(), "data"),  
    data_viz_dir = os.path.join(get_base_dir(), "data_viz"),  
    device_type = "vive",  
    device_params = {  
        "calib_uvw": [-1.5901484677973787, -3.130982168874227, 1.1842360521284816],  
    }  
)
```

Setting up configurations for real-world

1. ROBOT_CONFIG

2. TASK_CONFIG

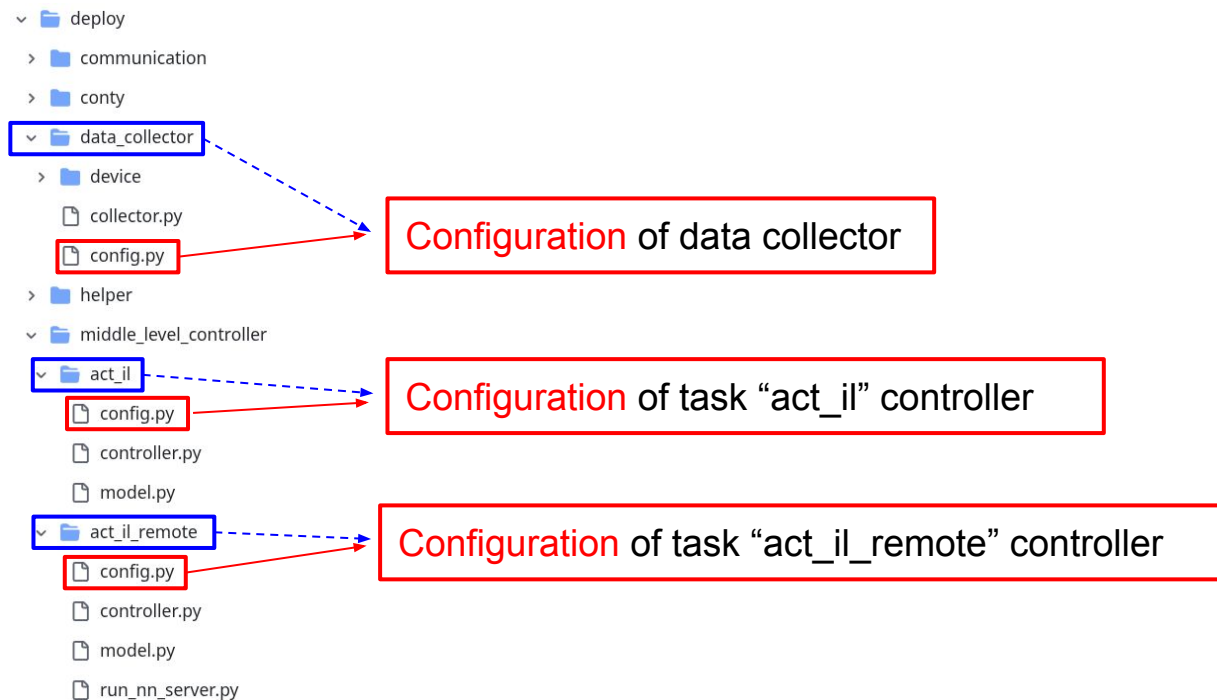
- Task name
- CAMERA_CONFIG
- MODEL_CONFIG
- DATA_CONFIG
- EXTRA_CONFIG
 - Extra task-specific configuration
 - Ex) start movement, finish movement, control post processing ..

```
@dataclass
class EXTRA_CONFIG:
    home_movement_fn: Callable = default_home_movement
    start_movement_fn: Callable = default_start_movement
    finish_movement_fn: Callable = default_finish_movement

    control_post_process_fn : Callable = lambda x: x
```

Setting up configurations for real-world

- **Configurations are intended to be defined for each task** by generating an instance from the base configuration.



Collect data

1. Calibrate vive
2. Setup configuration
3. Collect data
 - *python collect_data.py*

```
DATA_COLLECTOR_TASK_CONFIG = TASK_CONFIG(  
    name = "pick_and_place", # Task name to collect data  
    camera_config = CAMERA_CONFIG(  
        cam_params = {  
            "left": {  
                "serial": "207222072747",  
                "enable_depth": False  
            },  
            "right": {  
                "serial": "317622073859",  
                "enable_depth": False  
            }  
        },  
    ),  
    model_config = None,  
    data_config = DATA_CONFIG(  
        device_type = "vive",  
        device_params = {  
            "calib_uvw": [-1.5901484677973787, -3.130982168874227, 1.1842360521284816],  
        },  
    ),  
)
```

Task name

No model yet

Calibration result

- ✓ deploy
 - > communication
 - > conty
 - ✓ data_collector
 - ✓ device
 - base.py
 - vive.py
 - collector.py
 - config.py
 - > helper
 - > middle_level_controller
 - > perception
 - > third_party/vive
 - .gitignore
 - README.md
 - collect_data.py
 - fsm_demo.py
 - nn_policy_unit_test.py
 - task_demo.py

Configuration of data collector



Collect data

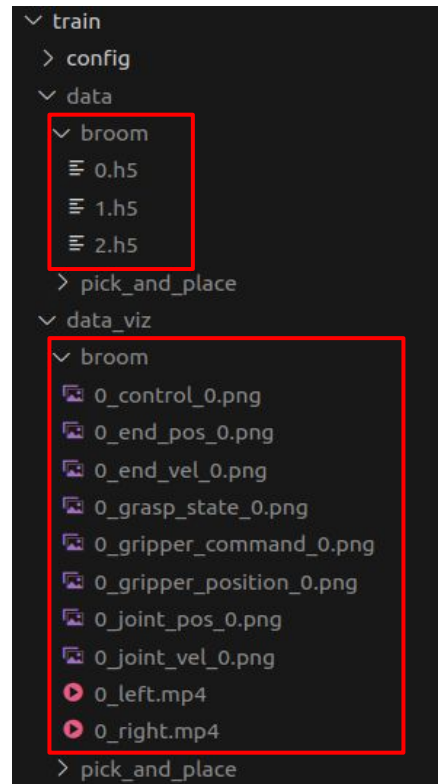
1. Calibrate vive
2. Setup configuration
3. Collect data

- `python collect_data.py`

Keyboard	Command	Description
1	MOVE_TO_TASK_HOME	Move to task-specific robot home position
2	EXECUTE_START_STATE_COLLECTION	Collect data after moving to robot home position
3	EXECUTE_CURRENT_STATE_COLLECTION	Collect data from current robot position



- High-level command is assigned by **keyboard**.
- During data collection,
 - Vive “Toggle down”: Record start/stop
 - Vive “Trigger”: Gripper open/close
 - After recording single episode, it will ask “Click 's' to save / Click 'e' to not save”
- By default, **raw data** and **visualized data** are saved in ***train/data/TASK*** and ***train/data_viz/TASK***.



Train model

1. Preprocess data

- `python preprocess.py --task TASK_NAME`

2. Setup configuration

3. Train

- `python imitate.py`
`--config-path=config/CONFIG_DIRECTORY`
`--config-name=CONFIG_FILE`

Training configurations

The codebase supports below configurations.

(1) Algorithm

- ACT (i.e., Action Chunking Transformer) [\[Paper\]](#)

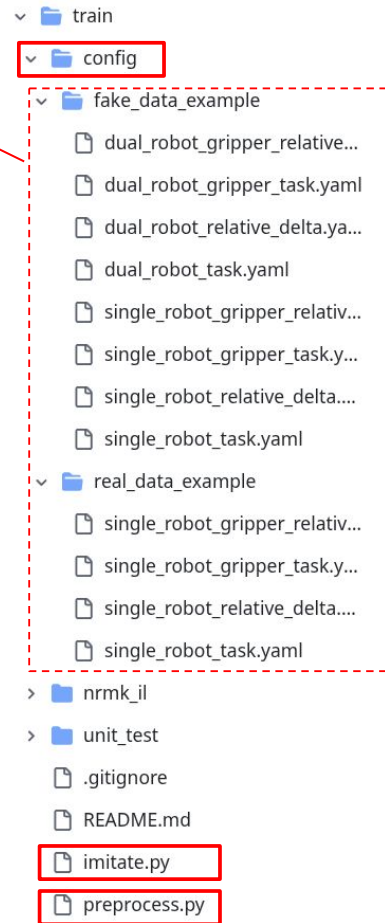
(2) Robot mode

- Single robot arm
- Single robot arm with gripper
- Dual robot arm
- Dual robot arm with gripper

(3) Control mode (i.e., output of neural network)

- Task space
- Relative delta task space [\[Paper\]](#)

Sample
configurations



Train model

1. Preprocess data
 - `python preprocess.py --task TASK_NAME`
2. Setup configuration
3. Train
 - `python imitate.py`
`--config-path=config/CONFIG_DIRECTORY`
`--config-name=CONFIG_FILE`

- **Success detector** can be trained concurrently.
- **Wandb logger** is supported to visualize training progress.
- Real-world data examples are provided in `unit_test/example/data`.

```
### Base configuration
base:
  policy_class: act
  task_name: broom
  seed: 0
  num_workers: 4
  logging: False # [Option] Can enable wandb logging

### Train configuration
train:
  num_epochs: 10000
  batch_size: 128
  success_detector_batch_size: 32
  lr: 1e-5
  lr_backbone: 1e-5
  weight_decay: 1e-4

### ACT configuration
policy:
  ## robot / control mode
  robot_mode: single_robot
  control_mode: task_space

  input_shapes:
    observation.images.rgb.left: [3, 224, 224]
    observation.images.rgb.right: [3, 224, 224]
    observation.qpos: [6]
    observation.qvel: [6]

  output_shapes:
    action.end_pos: [3]
    action.end_ori: [6]
    is_success: [1] # [Option] Can train success detector concurrently

  chunk_size: 20 # 1s
```

Sample configuration

Deploy model

1. Add task controller

- Add folder under *middle_level_controller*
- Add *config.py*, *model.py*, *controller.py*

2. Set task name

3. Run controller

- *python task_demo.py*

- **Controllers are intended to be defined for each task** due to potential differences between task and model design.
- We offer a **general controller implementation** to deploy any task trained with **ACT** in “act_il”.

- ✓ deploy
- > communication
- > conty
- > data_collector
- > helper

```
class NN_controller(Base_NN_controller):  
    ROBOT_CONFIG = CUSTOM_ROBOT_CONFIG  
    ROBOT_IDS = CUSTOM_ROBOT_CONFIG.robot_ids  
    TASK_CONFIG = CUSTOM_TASK_CONFIG  
    TASK_NAME = CUSTOM_TASK_CONFIG.name  
  
    def __init__(self, robot: Dict[int, Robot] | None = None, **kwargs):
```

```
class NN_policy(Empty_NN_policy):  
    """  
    Policy with loaded weights  
    """  
    def __init__(self, robot config: ROBOT_CONFIG, task config: TASK_CONFIG):
```

Task name

- ✓ middle_level_controller
 - ✓ act_il
 - config.py
 - controller.py
 - model.py
 - > act_il_remote
- > perception
- > third_party/vive
 - .gitignore
 - README.md
 - collect_data.py
 - fsm_demo.py
 - nn_policy_unit_test.py
 - task_demo.py

Configuration

NN_controller:
Policy wrapper
connecting model to
robots and sensors

NN_Policy:
Model wrapper including
inference, preprocess ..

Deploy model

1. Add task controller
 - Add folder under *middle_level_controller*
 - Add *config.py*, *model.py*, *controller.py*
2. Set task name
3. Run controller
 - *python task_demo.py*

Unit test to **debug** implementation of
NN policy without robot and sensor connection

Task name

- ✓ deploy
- > communication
- > conty
- > data_collector
- > helper

- ✓ middle_level_controller
 - ✓ act_il
 - config.py
 - controller.py
 - model.py
 - > act_il_remote

- > perception
- > third_party/vive
- .gitignore
- README.md
- collect_data.py
- fsm_demo.py
- nn_policy_unit_test.py
- task_demo.py

```
class NN_controller(Base_NN_controller):  
    ROBOT_CONFIG = CUSTOM_ROBOT_CONFIG  
    ROBOT_IDS = CUSTOM_ROBOT_CONFIG.robot_ids  
    TASK_CONFIG = CUSTOM_TASK_CONFIG  
    TASK_NAME = CUSTOM_TASK_CONFIG.name  
  
    def __init__(self, robot: Dict[int, Robot] | None = None, **kwargs):
```

```
class NN_policy(Empty_NN_policy):  
    """  
    Policy with loaded weights  
    """  
    def __init__(self, robot config: ROBOT_CONFIG, task config: TASK_CONFIG):
```

NN_Policy:
Model wrapper including
inference, preprocess ..

Deploy model

1. Add task controller

- Add folder under *middle_level_controller*
- Add *config.py*, *model.py*, *controller.py*

2. Set task name

3. Run controller

- *python task_demo.py*

- High-level command is assigned by **keyboard**.
- We support **DAGGER**, which allows a human to intervene when the model enters a potential failure mode and **collect additional data for retraining**.

Keyboard	Command	Description
1	MOVE_TO_TASK_HOME	Move to task-specific robot home position
2	EXECUTE_TASK	Run task controller
0	EXECUTE_NN_STOP	Stop task controller
3	EXECUTE_START_STATE_DAGGER	Stop task controller and collect data after moving to robot home position
4	EXECUTE_CURRENT_STATE_DAGGER	Stop task controller and collect data from current robot position

```
if __name__ == "__main__":
    # Set task name
    demo_task = "act_il"
    # demo_task = "act_il_remote"

    # Set nn controller and robot connection
    NN_controller = load_NN_controller(controller_type=demo_task)
    nn_controller: Base_NN_controller = NN_controller()

    # Load nn model weights
    nn_controller.load_policy()

    # Check DAGGER and set data_collector if necessary
    if nn_controller.TASK_CONFIG.data_config is not None:
        #####
        ## DAGGER enabled ##
        #####
        import importlib
        from helper.controller_utils import Controller
        from helper.config_utils import ROBOT_CONFIG, TASK_CONFIG
        from data_collector.collector import DataCollectionScheduler
        from data_collector.config import DATA_COLLECTOR_ROBOT_CONFIG, DATA_COLLECTOR_TASK_CONFIG

        # Overwrite robot and task configurations for data collector
        module = importlib.import_module(f"middle_level_controller.{demo_task}.config")
        DataCollectionScheduler.ROBOT_CONFIG = module.CUSTOM_ROBOT_CONFIG
        DataCollectionScheduler.TASK_CONFIG = module.CUSTOM_TASK_CONFIG

        data_collection_scheduler: Controller = DataCollectionScheduler(
            robot=nn_controller.robot,
            camera=nn_controller.camera,
            dagger_mode=True
        )
    else:
        #####
        ## DAGGER disabled ##
        #####
        data_collection_scheduler = None
```

(Extra 1) Deploy model with server-client

1. Implement model server
 - Add *run_nn_server.py* and implement *RequestHandler*
2. Add dependencies to the third-party model's conda environment.
 - *pip install pyzmq*
3. Run model server
 - *python middle_level_controller/TASK/run_nn_server.py*
4. Run robot client
 - *python task_demo.py*

```
class RequestHandler(BaseRequestHandler):
    def __init__(self):
        self.policy: ACTPolicy | None = None

    def handle(self, received data):
```

```
def run_server(args):
    #####
    # External nn model specific code
    # In this example, the external nn model is ACT #
    #####
    request_handler = RequestHandler()
    server = PickleServer(host="localhost", port=args.port, request_handler=request_handler)
    server.serve()
```

```
class NN_policy(Empty_NN_policy):
    """
    Policy with loaded weights
    """
    def __init__(self, robot_config: ROBOT_CONFIG, task_config: TASK_CONFIG):
        super(NN_policy, self).__init__(robot_config=robot_config, task_config=task_config)

    # Set client to communicate with the nn server
    self.client = PickleClient(host="localhost", port=self.task_config.model_config.port)
```

- To enable **evaluating third-party models** beyond ACT **without conda environment conflicts**, we provide a **server-client example** in *middle_level_controller/act_il_remote*.
- We provide example with ACT in “*act_il_remote*”.

(Extra 2) Chaining multiple controllers with FSM

```
class FSM(StateMachine):
    starting = State(initial=True)
    brooming = State()
    pick_and_placing = State()
    finishing = State()

    start = (
        starting.to(brooming, cond="target_reached")
        | starting.to(finishing)
    )

    broom = (
        brooming.to(pick_and_placing, cond="task_succeed")
        | brooming.to(brooming, cond="task_failed")
        | brooming.to.itself(internal=True)
    )

    pick_and_place = (
        pick_and_placing.to(finishing, cond="task_succeed")
        | pick_and_placing.to(pick_and_placing, cond="task_failed")
        | pick_and_placing.to.itself(internal=True)
    )
```

- As imitation learning controllers often **struggle with long-horizon tasks of multiple sub-tasks**, we provide a simple example of chaining controllers with a **Finite-State Machine** in *fsm_demo.py*.

- ✓  deploy
 - >  communication
 - >  conty
 - >  data_collector
 - >  helper
- ✓  middle_level_controller
 - ✓  act_il
 -  config.py
 -  controller.py
 -  model.py
 - >  act_il_remote
- >  perception
- >  third_party/vive
 -  .gitignore
 -  README.md
 -  collect_data.py
 -  fsm_demo.py
 -  nn_policy_unit_test.py
 -  task_demo.py

(Extra 3) Deploy model with Neuromeka Conty

1. Start MoveMimicServicer
 - `conty/run_mimic_servicer.ipynb`
2. In Conty, go to the Program → MoveMimic tab and follow the sequence

- ✓ **deploy**
 - > **communication**
 - ✓ **conty**
 - > **grpc**
 - `mimic_servicer.py`
 - `run_mimic_servicer.ipynb`
 - > **data_collector**
 - > **helper**
 - > **middle_level_controller**
 - > **perception**
 - > **third_party/vive**
 - `.gitignore`
 - `README.md`
 - `collect_data.py`
 - `fsm_demo.py`
 - `nn_policy_unit_test.py`
 - `task_demo.py`

